

RQL: A Portable Intermediate Representation for Retrieval Plans in RAG Systems

Incremental research draft — not a camera-ready submission

Vijay Kumar

Research draft, Europe/Dublin

`vijaykumarjob0701/rql-rag-query-language`

16 September 2026

Draft status. Structured for growth into a citable article/thesis. Claims are labelled [ESTABLISHED], [HYPOTHESIS], or [PROVISIONAL]. No fabricated experimental results. Not claimed ready for NeurIPS, VLDB, or arXiv acceptance.

Abstract

Retrieval-augmented generation (RAG) systems compose embedding search, lexical retrieval, metadata and ACL filters, hybrid fusion, reranking, and sometimes graph or multi-hop expansion. Today those stages are usually glued together in application code against vendor-specific vector-database APIs. This draft proposes **RQL (Retrieval Query Language)** not merely as “SQL for vectors”, but as a **portable intermediate representation for retrieval plans**: a small algebra over scored evidence, plus cost-based physical planning under recall, latency, and ACL constraints, compiling through adapters to heterogeneous backends.

We separate [ESTABLISHED] survey material from [HYPOTHESIS] original design and from [PROVISIONAL] literature pending multimodal re-read. Careful multimodal passes over ACORN [Patel et al., 2024], VBASE [Zhang et al., 2023], and Filtered-DiskANN [Gollapudi et al., 2023] ground filtered-ANN physical strategies (SUBGRAPH, ITERATIVE, SPECIALIZED); Cormack et al. [Cormack et al., 2009] grounds $Fuse_{rrf}$ as established rank-fusion prior art. We define an evaluation *plan* and reproducibility layout; we do **not** report unreproduced benchmark scores.

1 Introduction

1.1 Problem

Modern RAG stacks must retrieve *evidence*—chunks, passages, entities, paths—under mixed modalities: dense vectors, sparse/BM25 channels, structured predicates (including ACLs), and increasingly late-interaction or graph expansion [Lewis et al., 2020]. Vector databases expose powerful but **non-portable** interfaces (JSON filters, GraphQL operators, SQL extensions, Redis commands, Cypher search). As a result, accuracy, latency, and reliability become properties of brittle Python glue rather than of inspectable plans.

1.2 Motivation (accuracy, latency, reliability)

Accuracy. Hybrid fusion (e.g. reciprocal rank fusion [Cormack et al., 2009]), over-fetch+rerank, and multi-query rewriting improve recall only when stages are explicit and tunable.

Latency. Filter placement (pre vs. post vs. subgraph vs. specialised label graphs vs. iterator ANN) and ANN parameters dominate end-to-end cost [Patel et al., 2024, Gollapudi et al., 2023, Zhang et al., 2023].

Reliability. Silent empty results, client-side ACL filtering, and irreproducible efSearch folklore are operational failures; plans with EXPLAIN and mandatory predicate pushdown targets are the remedy [HYPOTHESIS].

1.3 Thesis statement

RQL is a portable IR for retrieval plans—an algebra over scored multisets of evidence together with a Cascades/Calcite-style physical planner that chooses filtered-ANN and fusion strategies under explicit recall/latency/ACL contracts, then compiles via polystore-like shims to concrete engines. The textual DSL is a frontend, not the contribution’s centre of mass [HYPOTHESIS].

1.4 Contributions (draft)

1. A clear problem formulation for portable retrieval plans (Section 4).
2. A hypothesized retrieval algebra and physical operator set (Sections 5–6) [HYPOTHESIS].
3. Compilation/adaptor doctrine inspired by Substrait and polystores (Section 7) [HYPOTHESIS].
4. An evaluation and reproducibility *plan* without fabricated metrics (Section 9).
5. Careful related-work positioning, with ACORN, VBASE, and Filtered-DiskANN multimodally re-read (Section 3).

2 Background

2.1 Approximate nearest neighbour search

Graph indices such as HNSW [Malkov and Yashunin, 2018] and disk-oriented graphs (DiskANN line) underpin production vector search. Search quality/latency is controlled by parameters (e.g. candidate list size), yielding recall–throughput curves rather than exact SQL cardinalities [ESTABLISHED].

2.2 Hybrid vector + predicate search

Applications need ANN jointly with structured predicates. Common strategies include **pre-filtering**, **post-filtering**, specialised low-cardinality equality indices, and **predicate-subgraph** traversal as in ACORN [Patel et al., 2024] [ESTABLISHED] (see Section 3).

2.3 Hybrid lexical–dense fusion

Reciprocal Rank Fusion (RRF) combines ranked lists without score calibration [Cormack et al., 2009] [ESTABLISHED]. Production engines also expose weighted linear fusion and learned rankers; choice is workload-dependent [PROVISIONAL].

2.4 Late interaction

ColBERT-style late interaction [Khattab and Zaharia, 2020] and subsequent engines/rewrites (e.g. MUVERA [Dhulipala et al., 2024]) show that multi-vector retrieval admits planner rewrites [PROVISIONAL] (pending dedicated multimodal reads).

2.5 RAG as a staged pipeline

A typical RAG retrieve path is: embed → search channels → filter → fuse → (rerank) → expand/traverse → generate. RQL aims to name these stages as plan operators rather than ad-hoc functions [HYPOTHESIS].

3 Related work

3.1 Vector database interfaces

Commercial and OSS systems expose vendor-specific query surfaces (metadata filters, hybrid search APIs, SQL extensions such as pgvector). Survey details and URLs live in the companion research package; none is a cross-vendor ANSI-SQL analogue for multi-stage RAG plans [ESTABLISHED] (landscape notes).

3.2 Filtered ANN / FANNS

ACORN (multimodal re-read). Patel et al. [Patel et al., 2024] argue that pre- and post-filtering are brittle under selectivity and query–predicate correlation, and that specialised equality indices constrain predicate cardinality. They propose denser, predicate-agnostic HNSW variants (ACORN- γ , ACORN-1) that *search the predicate subgraph* to approximate an oracle HNSW over surviving vectors. Our Pass 1–5 notes (journal 0006; OKF bundle under `knowledge/reads/acorn-2403.04871/`) treat author QPS claims as **not reproduced here** [ESTABLISHED] for the problem taxonomy; [PROVISIONAL] for unreproduced numeric speedups.

Filtered-DiskANN (multimodal re-read). Gollapudi et al. [Gollapudi et al., 2023] change *graph construction* (FilteredVamana / StitchedVamana) so edges respect geometry and labels, then search with label-aware greedy expansion. Pass 1–5 notes (journal 0010; `knowledge/reads/filtered-diskann-www23/`) ground FilterExec = SPECIALIZED for equality/label predicates; complex SQL-like filters remain an author-stated open problem [ESTABLISHED] for the specialised-graph idea; [PROVISIONAL] for unreproduced QPS curves.

VBASE (multimodal re-read). Zhang et al. [Zhang et al., 2023] formalise *relaxed monotonicity* and expose ANN traversal as Volcano Open/Next/Close inside PostgreSQL, enabling filter-during-traversal, multi-column TopK, range filters, and vector joins without a predicted over-fetch K' . Pass 1–5 notes (journal 0009; `knowledge/reads/vbase-osdi23/`) ground FilterExec = ITERATIVE and a concrete path for VSimJoin [ESTABLISHED] for the iterator architecture; [PROVISIONAL] for unreproduced latency claims.

3.3 Rank fusion (RRF)

Cormack et al. (multimodal re-read). Cormack, Clarke, and Büttcher [Cormack et al., 2009] define Reciprocal Rank Fusion as $\text{RRFscore}(d) = \sum_{r \in R} 1/(k + r(d))$ with $k = 60$ fixed after a pilot study. RRF is unsupervised and *rank-only* (no requirement that channel scores be calibrated), unlike CombMNZ; Condorcet Fuse is the pairwise-majority comparator. Pass 1–5 notes (journal 0011; `knowledge/reads/rrf-cormack-sigir09/`) treat the **formula and default k** as [ESTABLISHED] IR prior art for logical Fuse_{rrf}; author TREC/LETOR MAP tables are **not reproduced here** [PROVISIONAL] as unreproduced empirics. Modern dense+BM25 product stacks already ship RRF; learned/linear fusion (e.g. Bruch-style CC) remains a separate sibling operator [HYPOTHESIS].

3.4 IR query languages

Classical IR languages (Indri, Galago, CQL/SRU) treat ranking as an operator algebra [met, 2004] [PROVISIONAL] (URL-verified references; deep re-read pending). RQL steals composability, not surface syntax fidelity [HYPOTHESIS].

3.5 Optimisers, portable plans, polystores

Apache Calcite [Begoli et al., 2018] and Substrait [sub, 2024] motivate logical/physical separation and portable plan IR. BigDAWG-style polystores [Duggan et al., 2015] motivate islands + shims [PROVISIONAL]. BlinkDB [Agarwal et al., 2013] motivates explicit approximate budgets [PROVISIONAL].

3.6 Knowledge packaging (methodology, not runtime)

Google’s Open Knowledge Format (OKF) packages knowledge as markdown concepts with YAML frontmatter and graph-shaped links [okf, 2025]. We use OKF-*shaped* bundles for research notes. We do **not** claim OKF is part of the RQL execution runtime [ESTABLISHED].

3.7 What RQL is not

RQL is not “another vendor filter DSL”, not a guarantee of identical recall across HNSW libraries, and not a claim that SQL alone solves multi-stage RAG [HYPOTHESIS].

4 Problem formulation

4.1 Setting

Let collections store entities with payloads, optional graph links, and one or more retrieval channels (dense vectors, lexical fields, multi-vectors). A **retrieval plan** consumes a user/agent query (text and/or vectors), mandatory security predicates, and optional budgets, and returns a ranked evidence multiset.

4.2 Requirements

1. **Portability:** one logical plan maps to multiple backends via adapters.
2. **Filter correctness:** ACL/tenant predicates are hard constraints (fail closed).
3. **Explicit approximation:** recall/latency targets are first-class, not hidden knobs.
4. **Composability:** fusion, rewrite, expand, traverse, and similarity joins are plan nodes.
5. **Observability:** logical/physical EXPLAIN with provenance for evaluation harnesses.

4.3 Non-goals (this draft)

Identical numeric recall across ANN implementations; full DDL standardisation; replacing embedding model APIs; unreproduced SOTA claims.

5 RQL algebra (original sketch)

All operators in this section are [HYPOTHESIS] unless marked otherwise.

5.1 Typed evidence relation

An **evidence relation** E is a multiset of tuples with at least:

$$E \subseteq Id \times Payload \times Score \times Channel \\ \times Provenance \times Attr^*.$$

Informal signature: each tuple carries a channel-local score until a fuse operator. Mandatory ACL attributes (tenant, clearance, ...) are part of $Attr$ and must remain enforceable in every physical plan [HYPOTHESIS].

Channels. $Channel \in \{\text{dense, bm25, late, graph, fused, } \dots\}$. Scores are comparable *within* a channel; cross-channel comparison requires Fuse.

5.2 Logical operators (kernel)

Operator	Signature (sketch)	Notes
$\text{Search}_{\text{dense}}(q, k)$	$\rightarrow E$	ANN leaf
$\text{Search}_{\text{bm25}}(q, k)$	$\rightarrow E$	Lexical leaf
$\text{Search}_{\text{late}}(q, k)$	$\rightarrow E$	Late interaction leaf
$\text{Filter}(P)$	$E \rightarrow E$	Predicates / ACL
Union	$E \times E \rightarrow E$	Multi-query
$\text{Fuse}_{\text{rrf}}(k_{\text{rrf}})$	$E^* \rightarrow E$	Formula $\sum 1/(k + r(d))$, default $k=60$ [ESTABLISHED] [Cormack et al., 2009] (journal 0011); RQL op name / compile [HYPOTHESIS]
$\text{Fuse}_{\text{linear}}(\alpha)$	$E^* \rightarrow E$	Weighted mix
$\text{Diversify}_{\text{mmr}}$	$E \rightarrow E$	Diversity
Rerank_m	$E \rightarrow E$	Cross-encoder etc.
Expand	$E \rightarrow E$	Parent/window
Rewrite	$q \rightarrow q^*$	HyDE / multi-query
Traverse_h	$E \rightarrow E$	Bounded graph hop
VSimJoin_θ	$E \times E \rightarrow E$	Similarity join

5.3 Rewrite examples (hypothesis, literature-grounded)

- **Post-filter brittleness (ACORN seed).** $\text{Filter}(P) \circ \text{Search}_{\text{dense}}(q, k)$ may be rewritten toward a physical SUBGRAPH or over-fetched POST plan when estimated selectivity / query–predicate correlation is adverse [Patel et al., 2024] [HYPOTHESIS] (choice is physical; logical Filter remains).
- **Iterator pushdown (VBASE seed).** When the adapter exposes Volcano-style ANN iteration, $\text{Filter}(P) \circ \text{Search}_{\text{dense}}$ may compile to $\text{FilterExec} = \text{ITERATIVE}$ (filter during Next) rather than TopK-then-drop [Zhang et al., 2023] [HYPOTHESIS].
- **Label-specialized graphs (Filtered-DiskANN seed).** Equality/label predicates with a capable filtered graph may compile to SPECIALIZED [Gollapudi et al., 2023] [HYPOTHESIS]; unsupported predicate shapes must not silently use that path.
- **Hybrid fuse (RRF seed).** Parallel $\text{Search}_{\text{dense}}$ and $\text{Search}_{\text{bm25}}$ (or other channels) remain separate until $\text{Fuse}_{\text{rrf}}(k_{\text{rrf}})$ with default $k_{\text{rrf}} = 60$ [Cormack et al., 2009] [ESTABLISHED] for the score; choosing RRF vs $\text{Fuse}_{\text{linear}}$ when scores are calibrated is physical/policy [HYPOTHESIS].
- **Similarity join.** $\text{VSimJoin}_\theta(E_1, E_2)$ may compile to nested $\text{Search}_{\text{dense}}$ / range-iterator joins when ITERATIVE (or equivalent) exists [Zhang et al., 2023] [HYPOTHESIS]; otherwise to blocked nested loops with explicit cost — not claimed implemented.

5.4 Semantics sketch

Channels remain separate until fuse. Mandatory predicates in $P_{\text{acl}} \subseteq P$ must be enforced by physical plans that cannot strip them client-side [HYPOTHESIS]. Rewrite operators carry explicit token/latency cost attributes for the optimiser.

5.5 Delineation from survey material

RRF [Cormack et al., 2009] (formula + $k=60$; journal 0011), HNSW [Malkov and Yashunin, 2018], ACORN’s predicate-subgraph idea [Patel et al., 2024], VBASE’s relaxed-monotonicity iterators [Zhang et al., 2023], and Filtered-DiskANN’s label-aware graphs [Gollapudi et al., 2023] are prior art (multi-modal notes: journals 0006, 0009, 0010, 0011). **Original to this draft:** packaging them as a single portable retrieval IR with explicit physical filter modes and budget contracts aimed at RAG reliability [HYPOTHESIS].

6 Physical planning (original sketch)

This section is [HYPOTHESIS] unless a citation marks established prior art.

6.1 Physical filter modes

Grounded in FANNS practice and multimodal re-reads of ACORN [Patel et al., 2024], Filtered-DiskANN [Gollapudi et al., 2023], and VBASE [Zhang et al., 2023]:

$\text{FilterExec} \in \{\text{PRE}, \text{POST}, \text{ITERATIVE}, \text{SUBGRAPH}, \text{SPECIALIZED}, \text{AUTO}\}.$

- PRE: restrict candidates then exact/ANN on survivors (high selectivity / small survivor set).
- POST: ANN then predicate; often needs over-fetch; brittle under low selectivity or low query–predicate correlation [Patel et al., 2024].
- SUBGRAPH: ACORN-like predicate-subgraph traversal when the backend capability exists [Patel et al., 2024].
- SPECIALIZED: label/equality-constrained graph indices (FilteredVamana / StitchedVamana class) when the predicate matches advertised capabilities [Gollapudi et al., 2023].
- ITERATIVE: Volcano-style Open/Next/Close ANN with early-stop (relaxed monotonicity class) enabling filter-during-traversal and join patterns [Zhang et al., 2023].
- AUTO: choose using stats (selectivity, specificity, correlation, empty-result risk); **never** violate ACL hard constraints.

Capability negotiation. Adapters must advertise which modes exist. Missing SUBGRAPH/SPECIALIZED/ITERATIVE forces fallback — the algebra must not pretend universal support [HYPOTHESIS].

6.2 Cost vector

Unlike classical selectivity-only costing, plans expose a vector:

1. Recall proxy (candidate depth, filter empty-result risk, low-specificity risk [Gollapudi et al., 2023]),
2. Latency (p50/p95 targets),
3. ACL safety (boolean hard constraint),
4. Token/dollar budget (embeddings, rerankers, rewrites).

BlinkDB-style explicit budgets [Agarwal et al., 2013] motivate surface options such as `RECALL TARGET / LATENCY` [HYPOTHESIS].

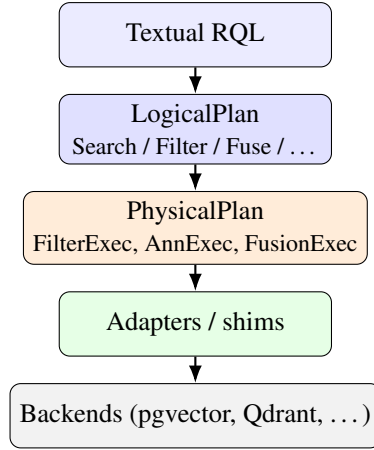


Figure 1: Hypothesized RQL compilation stack: text $\rightarrow$ logical plan $\rightarrow$ physical plan $\rightarrow$ adapters.

6.3 Fusion and rewrite rules

Examples **[HYPOTHESIS]**: independent channel searches $\rightarrow$ parallel schedule; Search_{late} without multi-vector support $\rightarrow$ optional FDE-style rewrite if available **[Dhulipala et al., 2024]**; low selectivity + POST $\rightarrow$ rewrite toward ITERATIVE/SUBGRAPH/SPECIALIZED/over-fetch depending on capabilities and predicate shape; VSimJoin $\rightarrow$ prefer iterator nested loops when ITERATIVE exists **[Zhang et al., 2023]**.

7 Compilation and adapters

[HYPOTHESIS] doctrine; Substrait/polystore inspiration is **[PROVISIONAL]**/**[ESTABLISHED]** as prior systems.

7.1 Layers

1. Parse textual RQL to `LogicalPlan`.
2. Optimise to `PhysicalPlan` with capability negotiation.
3. Emit backend calls through adapters (pgvector SQL, Qdrant query API, Pinecone, Neo4j, ...).

7.2 Capability negotiation

Profiles declare required features (hybrid+RRF, graph hop, late interaction). Missing capabilities yield either a safe rewrite, a shim CAST (e.g. client-side fuse—explicitly marked expensive/unsafe for ACL), or a hard error **[HYPOTHESIS]**.

7.3 Substrait adjacency

Substrait **[sub, 2024]** motivates a binary/portable plan encoding across engines. RQL may eventually emit a retrieval-oriented dialect or embed custom extensions; this draft does not claim a finished Substrait specification **[HYPOTHESIS]**.

8 Applications to RAG patterns

Patterns below map to example programs in the companion repo (`examples/*.rql`).

- Dense + ACL metadata filter (reliability-first pushdown).
- **Hybrid BM25+dense with RRF**: $\text{Fuse}_{rrf}(k=60)$ over channel-local rankings [Cormack et al., 2009] [ESTABLISHED] as the portable default when scores are incomparable; optional cross-encoder Rerank afterward (examples/02-hybrid-rrf.rql). Prefer Fuse_{linear} / learned mix when calibrated scores exist [HYPOTHESIS] (examples/03-hybrid-linear-weights.rql, 12-fuse-learned-explain.rql).
- Multi-query / HyDE-style rewrite as plan nodes (costed) [PROVISIONAL].
- Parent expansion and bounded multi-hop graph retrieval [PROVISIONAL].
- Late-interaction search with optional planner rewrite [PROVISIONAL].

RQL’s value is making these patterns *comparable* under EXPLAIN and shared eval harnesses [HYPOTHESIS] — not inventing RRF itself [ESTABLISHED].

9 Evaluation plan (no fabricated results)

*This section specifies **what we will measure**, not results we have not run.*

9.1 Datasets (planned)

- Retrieval: BEIR/MS MARCO-style collections as available under license; synthetic metadata predicates for filter selectivity sweeps.
- Filtered-ANN microbenchmarks aligned with published FANNS setups (e.g. SIFT-scale vectors with attribute predicates), following protocols from prior work such as ACORN [Patel et al., 2024]—**re-run**, do not copy tables.

HUMAN_TODO: dataset download, licenses, and digests — see experiments/HUMAN_TODO.md and experiments/protocols/02-datasets.md.

9.2 Metrics

Recall@ k , nDCG@ k , latency p50/p95, throughput (QPS) where applicable, **plan stability** (same RQL → same logical plan hash), and ACL violation count (must be zero).

9.3 Baselines

1. Hand-written SDK glue (LangChain/LlamaIndex-style scripts).
2. Vendor filter+hybrid APIs directly.
3. pgvector SQL hybrid patterns.
4. Fixed FILTER_MODE ablations vs AUTO (once implemented).

9.4 Ablation matrix (planned)

Channels $\in \{\text{dense, bm25, both}\}$; fuse $\in \{\text{RRF, linear}\}$; filter mode $\in \{\text{PRE, POST, AUTO}\}$; rerank on/off; rewrite on/off.

9.5 What exists today (agent-run)

- Planned-run printer: `experiments/harness/print_planned_runs.py` (emits `score=NOT_RUN` only).
- Deterministic RRF unit test on fixed lists: `experiments/harness/test_rrf_fusion.py`
→ `experiments/results/rrf_unit_test.txt`.

No end-to-end ANN, RAG, or human-judgment metrics are claimed.

9.6 What requires a human (Vijay)

GPU/CPU FANNS microbench, production/docker vector DB smokes, paid APIs, large datasets, and relevance judgments are listed as **HUMAN_TODO** items with exact protocols and expected in-repo artifact paths in `experiments/HUMAN_TODO.md`. Those results should be pushed to this same GitHub repository when available.

10 Threats to validity

- **Literature coverage:** only ACORN has completed multimodal Pass 1–5 in-repo; other citations remain **[PROVISIONAL]**.
- **ANN nondeterminism:** recall is implementation-dependent; portable IR cannot claim bit-identical results.
- **Adapter impedance:** not all backends expose subgraph or hybrid fusion; shims may hide costs.
- **Optimism bias:** algebra elegance does not guarantee planner quality without measured cost models.

11 Conclusion

We framed RQL as a **portable retrieval-plan IR** for RAG—algebra plus physical planning under recall/latency/ACL—rather than as cosmetic SQL for vectors **[HYPOTHESIS]**. ACORN’s multimodal re-read **[ESTABLISHED]** the need to treat filter strategy as a first-class physical decision **[Patel et al., 2024]**. The immediate path is: finish Pass 1–5 on VBASE and Filtered-DiskANN, freeze a LogicalPlan schema, implement two adapters, and execute the evaluation plan with published artifacts—without shortcutting integrity.

A Reproducibility checklist

1. Scripts and configs under `experiments/` with pinned dependency files when benchmarks are added.
2. Dataset license notes and download digests (**HUMAN_TODO**: `experiments/protocols/02-datasets.r`).
3. Seeded RNGs where stochasticity exists; report ANN library versions in `ENV.txt`.
4. Store logical/physical plan JSON beside metrics (no orphaned scores).
5. Separate AUTHOR_CLAIM citations from REPRODUCED runs in result tables.
6. Research notes: journal + OKF-shaped bundles for paper reads.
7. Hand-off list for compute/credentials beyond the agent sandbox: `experiments/HUMAN_TODO.md`.

References

- Indri query language reference. <http://lemurproject.org/lemur/IndriQueryLanguage.php>, 2004. Classical IR language documentation; deep re-read pending.
- Substrait: Cross-language serialization for relational algebra. <https://substrait.io/>, 2024. Project site; accessed from research notes 2026-09-16.
- Open knowledge format (OKF). Google Cloud Platform, GitHub, 2025. URL <https://github.com/GoogleCloudPlatform/open-knowledge-format>. Also: <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing>.
- Sameer Agarwal et al. BlinkDB: Queries with bounded errors and bounded response times on very large data. In *EuroSys*, 2013. URL <https://arxiv.org/abs/1203.5485>. Author list abbreviated in draft BibTeX.
- Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. *arXiv preprint arXiv:1802.10233*, 2018. URL <https://arxiv.org/abs/1802.10233>.
- Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, Boston, Massachusetts, USA, 2009. ACM. doi: 10.1145/1571941.1572114. URL <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>.
- Laxman Dhulipala, Majid Hadian, Rajesh Jayaram, Jason Lee, and Vahab Mirrokni. MUVERA: Multi-vector retrieval via fixed dimensional encodings. *arXiv preprint arXiv:2405.19504*, 2024. URL <https://arxiv.org/abs/2405.19504>. Also NeurIPS 2024 per Google Research publication page; venue details verify before camera-ready.
- Jennie Duggan et al. The BigDAWG polystore system. *SIGMOD Record*, 2015. URL https://sigmod.org/publications/sigmodRecord/1506/pdfs/04_vision_Duggan.pdf. Author list abbreviated in draft BibTeX.
- Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premukumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-diskANN: Graph algorithms for approximate nearest neighbor search with filters. In *WWW*, 2023. URL <https://harsha-simhadri.org/pubs/Filtered-DiskANN23.pdf>. Multimodal Pass 1–5: journal 0010; OKF knowledge/reads/filtered-diskann-www23/.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of SIGIR*, 2020. URL <https://arxiv.org/abs/2004.12832>.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020. URL <https://arxiv.org/abs/2005.11401>. Author list abbreviated in draft BibTeX; verify full authors before camera-ready.
- Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018. URL <https://arxiv.org/abs/1603.09320>.

Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *arXiv preprint arXiv:2403.04871*, 2024. URL <https://arxiv.org/abs/2403.04871>.

Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *OSDI*, pages 377–395, 2023. URL https://www.usenix.org/system/files/osdi23-zhang-qianxi_1.pdf. Multimodal Pass 1–5: journal 0009; OKF knowledge/reads/vbase-osdi23/.